

ISLAMIC UNIVERSITY OF TECHNOLOGY



MACHINE LEARNING

CSE 4621

Logistic Regression & Gradient Descent

Author:

Ishmam Tashdeed

Lecturer

IUT, CSE

Contents

1	Logistic Regression	2
1.1	Why Not Linear Regression for Classification?	2
1.2	The Sigmoid Function	2
1.3	The Logistic Regression Model	3
2	The Loss Function: Binary Cross-Entropy	3
2.1	Derivation from Maximum Likelihood	3
3	Gradient Descent for Logistic Regression	4
3.1	Gradient Descent: The Big Picture	4
3.2	Computing the Gradient via the Chain Rule	4
3.3	Gradient over the Full Dataset	5
4	Regularization	6
4.1	The Problem of Overfitting	6
4.2	L2 Regularization (Ridge)	6
4.3	L1 Regularization (Lasso)	7

1 Logistic Regression

1.1 Why Not Linear Regression for Classification?

Before introducing logistic regression, let us understand why we cannot simply use linear regression when the output y is a class label (e.g., 0 or 1).

Linear regression predicts a real number $\hat{y} = \boldsymbol{\theta}^\top \mathbf{x}$, which can be any value in $(-\infty, +\infty)$. But for a binary classification problem, we want a *probability* between 0 and 1. A raw linear output can easily exceed this range, making it meaningless as a probability.

We therefore need a function that squashes any real number into the interval $(0, 1)$. That function is the **sigmoid**.

1.2 The Sigmoid Function

Definition 1.1 (Sigmoid Function). The **sigmoid** (or *logistic*) function $\sigma : \mathbb{R} \rightarrow (0, 1)$ is defined as:

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Some important properties of the sigmoid:

- $\sigma(x) \in (0, 1)$ for all $x \in \mathbb{R}$.
- $\sigma(0) = 0.5$ — the function is centered at the origin.
- $\sigma(x) \rightarrow 1$ as $x \rightarrow +\infty$ and $\sigma(x) \rightarrow 0$ as $x \rightarrow -\infty$.
- Its derivative has a clean closed form: $\sigma'(x) = \sigma(x)(1 - \sigma(x))$.

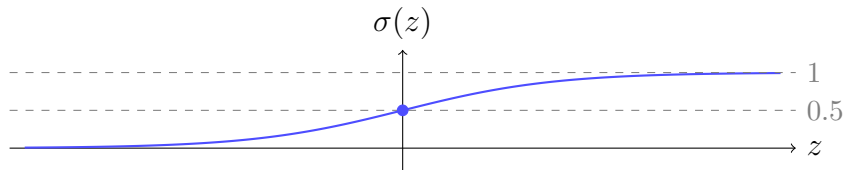


Figure 1: The sigmoid function maps any real number to the interval $(0, 1)$.

1.3 The Logistic Regression Model

In logistic regression, we model the *probability* that a data point belongs to class 1 given its features $\mathbf{x}$:

$$\hat{y} = P(y = 1 \mid \mathbf{x}; \boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}^\top \mathbf{x}) = \frac{1}{1 + e^{-\boldsymbol{\theta}^\top \mathbf{x}}},$$

where $\boldsymbol{\theta} \in \mathbb{R}^d$ is the weight vector and $\mathbf{x} \in \mathbb{R}^d$ is the feature vector.

Note

Throughout this note, we absorb the bias term θ_0 into the weight vector $\boldsymbol{\theta}$ by appending a constant feature $x_0 = 1$ to every data point. This keeps the notation clean without losing any generality.

The predicted class label is then obtained via a threshold:

$$\hat{y}_{\text{class}} = \begin{cases} 1 & \text{if } \hat{y} \geq 0.5, \\ 0 & \text{otherwise.} \end{cases}$$

Since $\sigma(z) \geq 0.5 \iff z \geq 0$, the decision boundary is the hyperplane $\boldsymbol{\theta}^\top \mathbf{x} = 0$.

2 The Loss Function: Binary Cross-Entropy

2.1 Derivation from Maximum Likelihood

The probabilistic model for a single binary label $y \in \{0, 1\}$ given $\hat{y} = \sigma(\boldsymbol{\theta}^\top \mathbf{x})$ is a Bernoulli distribution:

$$P(y \mid \mathbf{x}; \boldsymbol{\theta}) = \hat{y}^y (1 - \hat{y})^{1-y}.$$

Taking the log-likelihood over a dataset of m examples and negating (to turn maximisation into minimisation) to get the loss:

Definition 2.1 (Binary Cross-Entropy Loss).

$$J(\boldsymbol{\theta}) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})],$$

where $\hat{y}^{(i)} = \sigma(\boldsymbol{\theta}^\top \mathbf{x}^{(i)})$.

Note

Intuitively, when $y^{(i)} = 1$ the loss penalises $\hat{y}^{(i)}$ being small (far from 1), and when $y^{(i)} = 0$ it penalises $\hat{y}^{(i)}$ being large (far from 0). The logarithm makes this penalty grow steeply as the prediction moves in the wrong direction.

3 Gradient Descent for Logistic Regression

3.1 Gradient Descent: The Big Picture

Gradient descent is an iterative optimisation algorithm. Starting from some initial parameters $\boldsymbol{\theta}$, we repeatedly move in the direction of the *negative gradient* of the loss:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}),$$

where $\alpha > 0$ is the **learning rate**. Taking small steps downhill eventually brings us to the minimum.

3.2 Computing the Gradient via the Chain Rule

We need $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$. Let us compute $\frac{\partial J}{\partial \theta_j}$ for a single training example $(\mathbf{x}, y)$ first, then generalise. Define the intermediate variables:

$$z = \boldsymbol{\theta}^\top \mathbf{x}, \quad \hat{y} = \sigma(z).$$

Step 1 — Derivative of the loss with respect to $\hat{y}$:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = - \left(\frac{y}{\hat{y}} - \frac{1-y}{1-\hat{y}} \right) = \frac{\hat{y} - y}{\hat{y}(1-\hat{y})}.$$

Step 2 — Derivative of $\hat{y}$ with respect to z (sigmoid derivative):

$$\frac{\partial \hat{y}}{\partial z} = \sigma'(z) = \sigma(z)(1 - \sigma(z)) = \hat{y}(1 - \hat{y}).$$

Step 3 — Derivative of z with respect to θ_j :

$$\frac{\partial z}{\partial \theta_j} = \frac{\partial}{\partial \theta_j} \boldsymbol{\theta}^\top \mathbf{x} = x_j.$$

Step 4 — Chain rule:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \theta_j} &= \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial \theta_j} \\ &= \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \cdot \hat{y}(1 - \hat{y}) \cdot x_j \\ &= (\hat{y} - y) x_j.\end{aligned}$$

The $\hat{y}(1 - \hat{y})$ terms cancel beautifully, which is a direct consequence of using the sigmoid together with the log loss.

3.3 Gradient over the Full Dataset

Averaging over all m training examples, the j -th component of the gradient is:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)}.$$

In vector form, this is compactly written as:

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \frac{1}{m} \mathbf{X}^\top (\hat{\mathbf{y}} - \mathbf{y}),$$

where $\mathbf{X} \in \mathbb{R}^{m \times d}$ is the dataset matrix (one row per example, one feature per column), $\hat{\mathbf{y}} \in \mathbb{R}^m$ is the vector of predicted probabilities, and $\mathbf{y} \in \mathbb{R}^m$ is the vector of true labels.

Key Takeaway

The gradient update rule for logistic regression is:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \frac{\alpha}{m} \mathbf{X}^\top (\hat{\mathbf{y}} - \mathbf{y}).$$

Structurally, this looks **identical to the gradient descent update for linear regression**. The only difference lies in how $\hat{\mathbf{y}}$ is computed.

4 Regularization

4.1 The Problem of Overfitting

When a model has many parameters relative to the number of training examples, it may *overfit*: memorising noise in the training data and failing to generalise to new examples. One remedy is **regularization**, which adds a penalty term $\Omega(\boldsymbol{\theta})$ to the loss that discourages the weights from becoming too large:

$$J_{\text{reg}}(\boldsymbol{\theta}) = J(\boldsymbol{\theta}) + \lambda \Omega(\boldsymbol{\theta}),$$

where $\lambda > 0$ is a hyperparameter controlling how strongly we penalise large weights.

Note

It is standard practice to **not** regularise the bias term θ_0 , since it only shifts the decision boundary and does not directly contribute to overfitting in the same way the feature weights do.

4.2 L2 Regularization (Ridge)

Definition 4.1 (L2 Regularization). *The L2 penalty adds the squared ℓ_2 norm of the weights:*

$$\Omega_{L2}(\boldsymbol{\theta}) = \frac{1}{2} \|\boldsymbol{\theta}\|_2^2 = \frac{1}{2} \sum_{j=1}^d \theta_j^2.$$

The regularised loss becomes:

$$J_{L2}(\boldsymbol{\theta}) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})] + \frac{\lambda}{2} \sum_{j=1}^d \theta_j^2.$$

Gradient with L2 regularization:

We differentiate the regularisation term with respect to θ_j :

$$\frac{\partial}{\partial \theta_j} \left(\frac{\lambda}{2} \sum_{k=1}^d \theta_k^2 \right) = \lambda \theta_j.$$

Adding this to the unregularised gradient gives:

$$\frac{\partial J_{L2}}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)} + \lambda \theta_j.$$

In vector form:

$$\nabla_{\boldsymbol{\theta}} J_{L2} = \frac{1}{m} \mathbf{X}^\top (\hat{\mathbf{y}} - \mathbf{y}) + \lambda \boldsymbol{\theta}.$$

The weight update rule therefore becomes:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \left[\frac{1}{m} \mathbf{X}^\top (\hat{\mathbf{y}} - \mathbf{y}) + \lambda \boldsymbol{\theta} \right] = (1 - \alpha\lambda) \boldsymbol{\theta} - \frac{\alpha}{m} \mathbf{X}^\top (\hat{\mathbf{y}} - \mathbf{y}).$$

Key Takeaway

Notice the factor $(1 - \alpha\lambda)$ in front of $\boldsymbol{\theta}$. Each update first *shrinks* (or decays) the weights by a small fraction before subtracting the gradient. For this reason, L2 regularization is also called **weight decay**. It pushes all weights towards zero but never all the way to zero.

4.3 L1 Regularization (Lasso)

Definition 4.2 (L1 Regularization). *The L1 penalty adds the ℓ_1 norm of the weights:*

$$\Omega_{L1}(\boldsymbol{\theta}) = \|\boldsymbol{\theta}\|_1 = \sum_{j=1}^d |\theta_j|.$$

The regularised loss becomes:

$$J_{L1}(\boldsymbol{\theta}) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})] + \lambda \sum_{j=1}^d |\theta_j|.$$

Gradient with L1 regularization:

The absolute value $|\theta_j|$ is not differentiable at $\theta_j = 0$. However, we use its **subgradient** everywhere:

$$\frac{\partial}{\partial \theta_j} |\theta_j| = \text{sign}(\theta_j) = \begin{cases} +1 & \text{if } \theta_j > 0, \\ -1 & \text{if } \theta_j < 0, \\ 0 & \text{if } \theta_j = 0. \end{cases}$$

Therefore:

$$\frac{\partial J_{L1}}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)} + \lambda \text{sign}(\theta_j).$$

In vector form (where sign is applied element-wise):

$$\nabla_{\boldsymbol{\theta}} J_{L1} = \frac{1}{m} \mathbf{X}^\top (\hat{\mathbf{y}} - \mathbf{y}) + \lambda \text{sign}(\boldsymbol{\theta}).$$

The weight update rule is:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \left[\frac{1}{m} \mathbf{X}^\top (\hat{\mathbf{y}} - \mathbf{y}) + \lambda \text{sign}(\boldsymbol{\theta}) \right].$$

Key Takeaway

Unlike L2, the L1 penalty adds a *constant-magnitude* push of $\pm\lambda$ to each weight, regardless of how large or small that weight is. This means weights can be pushed all the way to **exactly zero**, effectively removing the corresponding feature from the model. L1 regularization therefore produces **sparse** solutions and acts as a form of automatic feature selection.

Note

The binary cross-entropy loss combined with the sigmoid activation yields a *convex* optimisation problem (as can be verified by showing its Hessian is positive semi-definite). This means, just like MSE for linear regression, gradient descent on logistic regression is guaranteed to converge to the *global* minimum, provided the learning rate α is chosen appropriately and the data are not perfectly linearly separable.